

СИСТЕМНОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

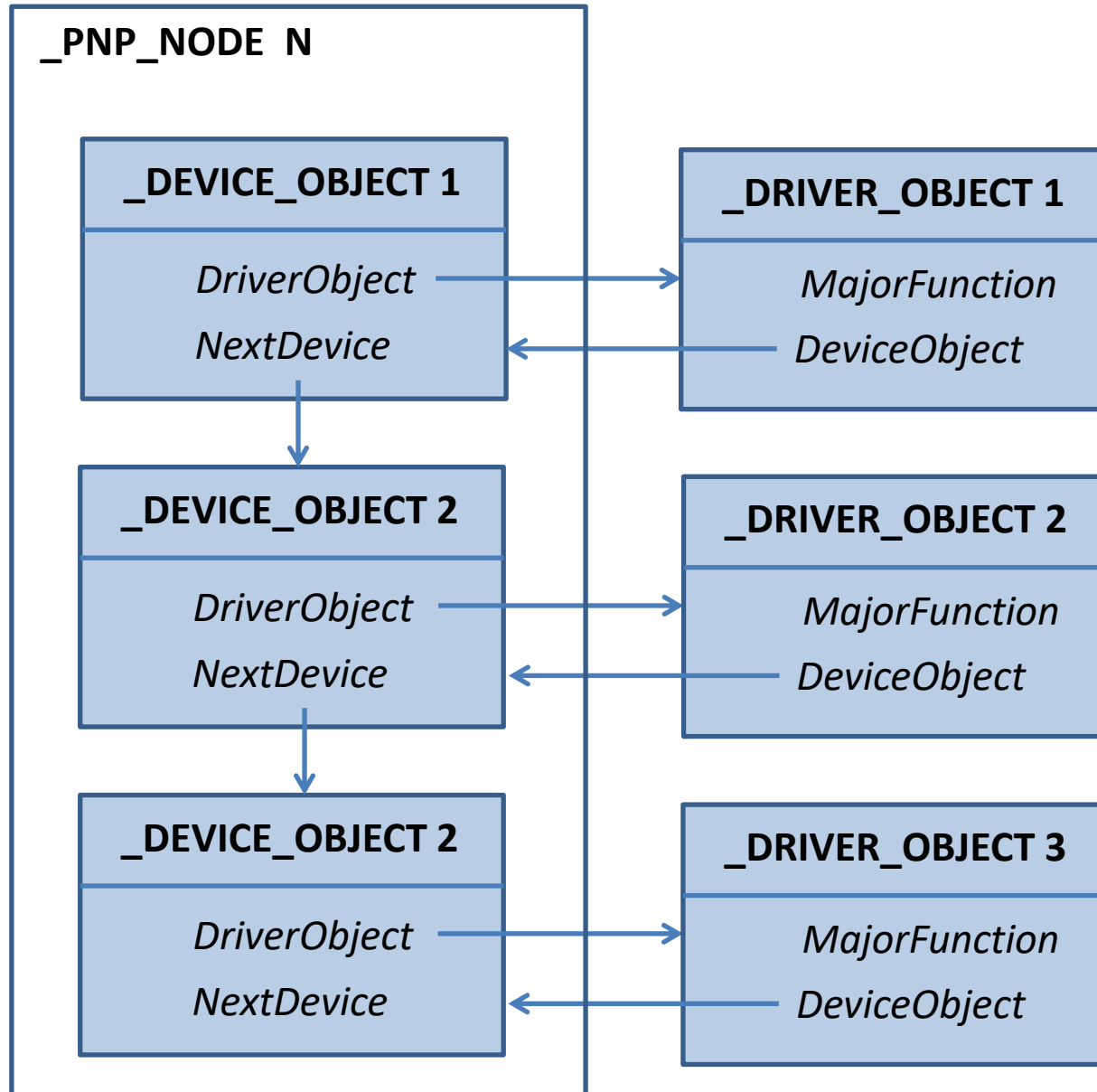
БЛОК 2

Ляхманов Д.А.
к.т.н., доцент каф. ИСУ

2018г.

СТРУКТУРА KMDF-ДРАЙВЕРА

РЕАЛИЗАЦИЯ PNP-ДЕРЕВА



CTPYKTYPA DRIVER_OBJECT

```
typedef struct _DRIVER_OBJECT
{
    CSHORT Type;
    CSHORT Size;
    PDEVICE_OBJECT DeviceObject;
    ULONG Flags;
    PVOID DriverStart;
    ULONG DriverSize;
    PVOID DriverSection;
    PDRIVER_EXTENSION DriverExtension;
    UNICODE_STRING DriverName;
    PUNICODE_STRING HardwareDatabase;
    PFAST_IO_DISPATCH FastIoDispatch;
    PDRIVER_INITIALIZE DriverInit;
    PDRIVER_STARTIO DriverStartIo;
    PDRIVER_UNLOAD DriverUnload;
    PDRIVER_DISPATCH
    MajorFunction[IRP_MJ_MAXIMUM_FUNCTION + 1];
} ;
```

CTPYKTYPA **DEVICE_OBJECT**

```
typedef struct _DEVICE_OBJECT
{
    CSHORT                Type;
    USHORT                Size;
    LONG                  ReferenceCount;
    struct _DRIVER_OBJECT *DriverObject;
    struct _DEVICE_OBJECT *NextDevice;
    struct _DEVICE_OBJECT *AttachedDevice;
    struct _IRP          *CurrentIrp;
    PIO_TIMER              Timer;
    ULONG                  Flags;
    ULONG                  Characteristics;
    __volatile PVPB        Vpb;
    PVOID                  DeviceExtension;
    DEVICE_TYPE           DeviceType;
    CCHAR                  StackSize;
    union
    {
        LIST_ENTRY         ListEntry;
        WAIT_CONTEXT_BLOCK Wcb;
    } Queue;
}
```

СТРУКТУРА ОБЪЕКТА УСТРОЙСТВА

```
ULONG           AlignmentRequirement;  
KDEVICE_QUEUE   DeviceQueue;  
KDPC            Dpc;  
ULONG           ActiveThreadCount;  
PSECURITY_DESCRIPTOR SecurityDescriptor;  
KEVENT          DeviceLock;  
USHORT          SectorSize;  
USHORT          Spare1;  
struct _DEVOBJ_EXTENSION * DeviceObjectExtension;  
PVOID           Reserved;  
} DEVICE_OBJECT, *PDEVICE_OBJECT;
```

ОБЯЗАТЕЛЬНЫЕ ФУНКЦИИ

Точка входа – функция, которая вызывается каждый раз при инициализации драйвера:

```
NTSTATUS DriverEntry(  
    _In_ struct _DRIVER_OBJECT *DriverObject,  
    _In_ PUNICODE_STRING RegistryPath );
```

DriverObject – объект драйвера, который содержит обработчики всех событий и будет регистрироваться в системе;

RegistryPath – строка, содержащая полный путь до ключа реестра, на основании которого регистрируется драйвер

ОБЯЗАТЕЛЬНЫЕ ФУНКЦИИ

Функция завершения драйвера вызывается, когда драйвер выгружается из оперативной памяти (напр. при отключении устройства):

```
VOID Unload(  
_In_ struct _DRIVER_OBJECT *DriverObject )  
{ ... }
```

DriverObject – зарегистрированный объект драйвера устройства

ОБЯЗАТЕЛЬНЫЕ ФУНКЦИИ

Функция `AddDevice` вызывается каждый раз, когда к PnP-дереву добавляется новое устройство:

```
NTSTATUS AddDevice(  
    _In_ struct _DRIVER_OBJECT *DriverObject,  
    _In_ struct _DEVICE_OBJECT *PhysicalDeviceObject )  
{ ... }
```

DriverObject – объект драйвера, который регистрируется в системе и содержит обработчики всех событий;

PhysicalDeviceObject – объект родительского устройства (PDO), созданный нижележащим драйвером

ПРИМЕР РЕАЛИЗАЦИИ ФУНКЦИИ DRIVERENTRY

```
NTSTATUS DriverEntry(
    _In_ PDRIVER_OBJECT pDriverObject,
    _In_ PUNICODE_STRING RegistryPath)
{
    for (int i = 0; i < IRP_MJ_MAXIMUM_FUNCTION; i++)
        pDriverObject->MajorFunction[i] = DispatchPassDown;

    pDriverObject->MajorFunction[IRP_MJ_READ] = DispatchRead;
    pDriverObject->DriverUnload = Unload;

    #ifdef DEBUG
        DbgPrint("Set DriverUnload function pointer...\n");
    #endif

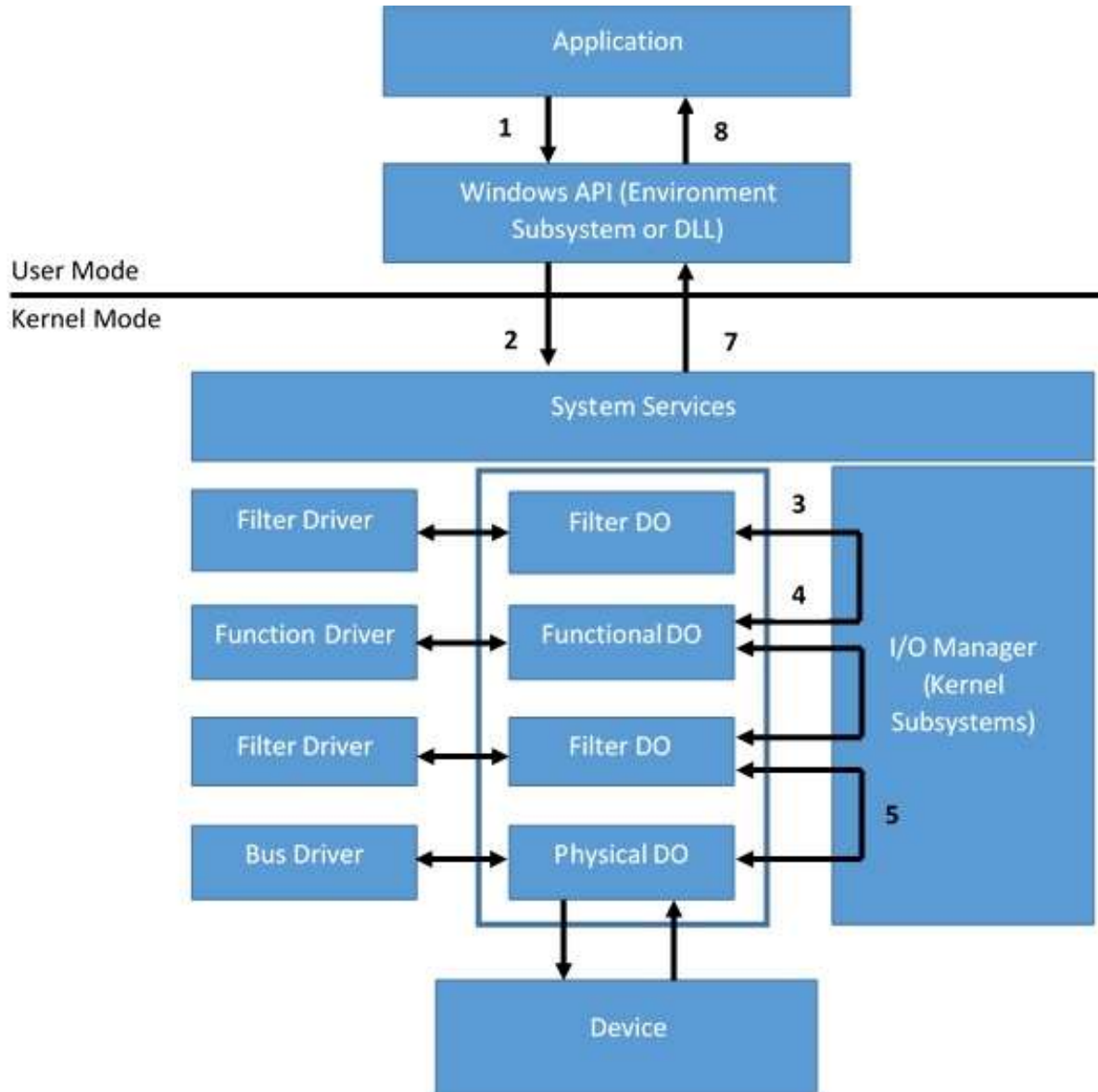
    return STATUS_SUCCESS;
}
```

ТАБЛИЦА ВЕКТОРОВ ОБЪЕКТА УСТРОЙСТВА

Dispatch routines:

[00] IRP_MJ_CREATE	ffffff880065d49d0	parport!PptDispatchCreateOpen
[01] IRP_MJ_CREATE_NAMED_PIPE	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[02] IRP_MJ_CLOSE	ffffff880065d4a78	parport!PptDispatchClose
[03] IRP_MJ_READ	ffffff880065d4bac	parport!PptDispatchRead
[04] IRP_MJ_WRITE	ffffff880065d4bac	parport!PptDispatchRead
[05] IRP_MJ_QUERY_INFORMATION	ffffff880065d4c40	parport!PptDispatchQueryInformation
[06] IRP_MJ_SET_INFORMATION	ffffff880065d4ce4	parport!PptDispatchSetInformation
[07] IRP_MJ_QUERY_EA	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[08] IRP_MJ_SET_EA	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[09] IRP_MJ_FLUSH_BUFFERS	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[0a] IRP_MJ_QUERY_VOLUME_INFORMATION	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[0b] IRP_MJ_SET_VOLUME_INFORMATION	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[0c] IRP_MJ_DIRECTORY_CONTROL	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[0d] IRP_MJ_FILE_SYSTEM_CONTROL	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[0e] IRP_MJ_DEVICE_CONTROL	ffffff880065d4be8	parport!PptDispatchDeviceControl
[0f] IRP_MJ_INTERNAL_DEVICE_CONTROL	ffffff880065d4c24	parport!PptDispatchInternalDeviceControl
[10] IRP_MJ_SHUTDOWN	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[11] IRP_MJ_LOCK_CONTROL	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[12] IRP_MJ_CLEANUP	ffffff880065d4af4	parport!PptDispatchCleanup
[13] IRP_MJ_CREATE_MAILSLLOT	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[14] IRP_MJ_QUERY_SECURITY	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[15] IRP_MJ_SET_SECURITY	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[16] IRP_MJ_POWER	ffffff880065d491c	parport!PptDispatchPower
[17] IRP_MJ_SYSTEM_CONTROL	ffffff880065d4d4c	parport!PptDispatchSystemControl
[18] IRP_MJ_DEVICE_CHANGE	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[19] IRP_MJ_QUERY_QUOTA	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[1a] IRP_MJ_SET_QUOTA	ffffff80001b6ecd4	nt!IopInvalidDeviceRequest
[1b] IRP_MJ_PNP	ffffff880065d4840	parport!PptDispatchPnp

ОБРАБОТКА ЗАПРОСОВ



ОБРАБОТЧИК IRP

Функция AddDevice вызывается каждый раз, когда к PnP-дереву добавляется новое устройство:

```
NTSTATUS DispatchIRP(  
    _In_ PDEVICE_OBJECT pDeviceObject,  
    _In_ PIRP pIrp  
) { ... }
```

pDriverObject – указатель на объект драйвера, который регистрируется в системе и содержит обработчики всех событий;

pIrp – указатель на структуру IRP, содержащей данные для обработки

CTPYKTYPА IRP

```
typedef struct _IRP
{
    PMDL                      MdlAddress;
    ULONG                     Flags;

    union
    {
        struct _IRP *MasterIrp;
        PVOID          SystemBuffer;
    } AssociatedIrp;

    IO_STATUS_BLOCK IoStatus;
    KPROCESSOR_MODE RequestorMode;
    BOOLEAN          PendingReturned;
    BOOLEAN          Cancel;
    KIRQL             CancelIrql;
    PDRIVER_CANCEL    CancelRoutine;
    PVOID             UserBuffer;
```

СТРУКТУРА IRP

```
union
{
    struct
    {
        union
        {
            KDEVICE_QUEUE_ENTRY DeviceQueueEntry;
            struct
            {
                PVOID DriverContext[4];
            };
        };
        PETHREAD Thread;
        LIST_ENTRY ListEntry;
    } Overlay;
} Tail;
} IRP, *PIRP;
```

ТАБЛИЦА ВЕКТОРОВ ОБЪЕКТА УСТРОЙСТВА

IRP_MJ_CREATE	– создание или открытие файла или устройства
IRP_MJ_CLOSE	– закрытие файла или устройства (напр. CloseFile)
IRP_MJ_QUERY_INFORMATION	– запрос данных о статусе устройства
IRP_MJ_SET_INFORMATION	– передача управляющей структуры на устройство
IRP_MJ_QUERY_EA	– запрос уточняющей информации с устройства
IRP_MJ_SET_EA	– изменение уточняющих данных устройства
IRP_MJ_FLUSH_BUFFERS	– сбросить все RAM-буферы и записать их на устройство
IRP_MJ_POWER	– посылается устройством или ОС при изменении режима питания устройств или выключении
IRP_MJ_FILE_SYSTEM_CONTROL	– используется для определения интерфейсов доступа к устройству
IRP_MJ_SHUTDOWN	– запрос отключения устройства
IRP_MJ_LOCK_CONTROL	– запрос доступности данных блочного устройства
IRP_MJ_CLEANUP	– посылается когда устройство завершило сессию доступа к данным
IRP_MJ_PNP	– посылается при перечислении устройства

СПАСИБО ЗА ВНИМАНИЕ